

Design and Implementations

The Evolution and Diversity of Programming Languages



Natural vs Artificial

Comparing programming languages with natural languages



Evolution

How programming languages have developed over time



Diversity

The range of situations programming languages must address

Programming Languages vs Natural Languages 🌐

Programming languages share traits with natural languages but differ fundamentally in their origin and definition.

Key Differences 🔍

Artificial Creation



Programming languages are artificial creations with clear and limited definitions, allowing them to be thoroughly defined and examined in their entirety.

Evolution Through Usage



Natural languages have evolved through usage and have changeable meanings as determined by their users in different groups.

```
// Example of precise definition in C
#include <stdio.h>

int main() {
    // In C, every statement has a precise meaning
    int x = 5; // Exactly 4 bytes allocated
    float y = 3.14; // Exactly 4 bytes with IEEE 754 format
    printf("x = %d, y = %f\n", x, y);
    return 0;
}
```



Real-World Example

While natural languages like English have words with multiple meanings that change over time (e.g., "cool" meaning both temperature and stylish), in C, the keyword "int" always means a signed integer of a specific size, with no ambiguity.

2/6

Evolution of Programming Languages



Programming languages have continuously evolved to meet new requirements and integrate with other languages.

Language Development

Several programming languages have been:

- Developed and created from scratch 
- Modified to fit new requirements 
- Integrated with others 
- Ultimately gone out of favor 

```
// Evolution from C to C++ - adding object-oriented features  
  
// Original C approach  
struct Point {  
    int x;  
    int y;
```

```
};

void init_point(struct Point *p, int x, int y) {
    p->x = x;
    p->y = y;
}

// C++ evolution with classes and methods
/*
class Point {
private:
int x, y;
public:
Point(int x, int y) : x(x), y(y) {}
int getX() const { return x; }
int getY() const { return y; }
};
*/
```



The Quest for a Universal Language

Although attempts have been made to create a single "global" programming language that suits all needs, none have been widely regarded as fulfilling this function. The diversity of programming needs necessitates a variety of languages.

3/6

Range of Programming Situations

The diversity of programming needs necessitates the development of varied programming languages.

Programming Diversity



Program Size – From little scripts created by a single hobbyist to large systems built by hundreds of programmers.



Programmer Skill Level – From novices who require simplicity above all else to specialists who are comfortable with significant complexity.



System Constraints – On systems ranging from microcontrollers to supercomputers, programs must balance speed, size, and simplicity.



Longevity – Programs can be developed once and not changed for generations, or their content can be modified on a regular basis.



Programmer Preferences – Programmers may be acclimated to addressing problems and representing them in a specific language.

```
// Different programming approaches for different situations

// Microcontroller code - optimized for size and speed
#include <avr/io.h>
void delay_ms(uint16_t ms) {
    while(ms--) {
        volatile uint16_t i = 1000;
        while(i--);
    }
}

// High-level application code - optimized for readability
#include <stdio.h>
int calculate_average(int arr[], int size) {
    int sum = 0;
    for (int i = 0; i < size; i++) {
        sum += arr[i];
    }
    return sum / size;
}
```

Abstraction in Programming Languages



One recurrent tendency in programming language evolution has been to offer a greater capacity to handle issues at a higher degree of abstraction.

Evolution of Abstraction

Programming languages have evolved from being closely tied to hardware to allowing programmers to express ideas that are further distant from straightforward translation to underlying hardware instructions.

```
// Evolution of abstraction in C

// Low-level approach - direct hardware manipulation
/*
// Direct memory access (older style)
char *video_memory = (char *)0xB8000;
video_memory[0] = 'H';
video_memory[1] = 0x07; // Color attribute
*/

// Higher-level abstraction - using standard libraries
#include <stdio.h>
int main() {
    printf("Hello, World!\n");
    return 0;
}

// Even higher-level abstraction - using functions
void print_greeting(char *name) {
    printf("Hello, %s!\n", name);
}

int main() {
    print_greeting("World");
    return 0;
}
```



Benefits of Abstraction

As programming languages have evolved to higher levels of abstraction, programmers have become less dependent on the computer's complexity. This allows them to perform more computation with less effort and write more functionality per time unit.

5/6

Natural-language Programming and Language Artifacts

Natural-language programming aims to eliminate specialized language requirements, but its merits are debated.

Approaches to Natural-language Programming



Criticism



Dijkstra and Perlis condemn natural-language programming, arguing that natural languages lack the precision needed for programming.

Hybrid Techniques



Structured English and SQL use hybrid techniques, combining natural language elements with formal programming constructs.

```
// Example of SQL - a hybrid natural/programming language

/*
-- Natural language-like query in SQL
SELECT name, age
FROM employees
WHERE department = 'Engineering'
ORDER BY age DESC;
*/

// Equivalent in C
#include <stdio.h>
#include <string.h>

typedef struct {
    char name[50];
    int age;
    char department[50];
} Employee;

void print_engineers_by_age(Employee employees[], int count) {
    // Sort employees by age (descending)
    for (int i = 0; i < count - 1; i++) {
        for (int j = 0; j < count - i - 1; j++) {
            if (employees[j].age < employees[j+1].age) {
                Employee temp = employees[j];
                employees[j] = employees[j+1];
                employees[j+1] = temp;
            }
        }
    }

    // Print engineers
    for (int i = 0; i < count; i++) {
        if (strcmp(employees[i].department, "Engineering") == 0) {
            printf("%s, %d\n", employees[i].name, employees[i].age);
        }
    }
}
}
```

Language Artifacts

The designers and users of a language must construct a variety of artifacts that control and facilitate programming practice. The most important of these artifacts are the language definition and implementation.